

# Capítulo 14: Design of Biological Systems

## Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

23 de junho de 2026

# Sumário

- 1 Introdução
- 2 Circuitos Genéticos Sintéticos
- 3 Engenharia Metabólica
- 4 Ferramentas de Design e Modelagem
- 5 Biologia Sintética de Células Inteiras
- 6 Desafios e Perspectivas
- 7 Exercícios
- 8 Referências

# Visão Geral do Capítulo

## Objetivos de Aprendizagem

- Compreender princípios de engenharia aplicados à biologia
- Projetar e analisar circuitos genéticos sintéticos
- Aplicar engenharia metabólica para produção de compostos
- Utilizar ferramentas computacionais de design
- Entender desafios e perspectivas da biologia sintética

## Biologia Sintética

Aplicação de princípios de engenharia para projetar e construir novos sistemas biológicos ou redesenhar sistemas naturais existentes.

# O que é Biologia Sintética?

## Definição: Biologia Sintética

Disciplina que combina biologia, engenharia e ciência da computação para projetar e construir novas funções e sistemas biológicos que não existem na natureza.

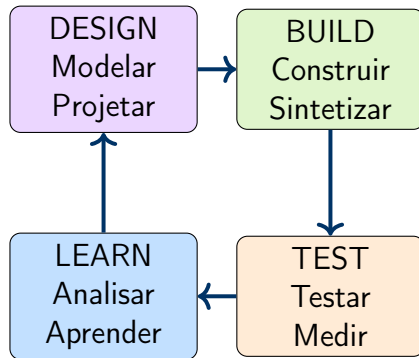
### Princípios de Engenharia:

- Modularidade
- Padronização
- Abstração
- Design racional
- Otimização

### Aplicações:

- Biocombustíveis
- Fármacos
- Biosensores
- Biomateriais
- Terapia celular
- Bioremediação

# Ciclo Design-Build-Test-Learn



**Importante!**

Processo iterativo: cada ciclo refina o design com base nos dados experimentais

## Considerações Éticas

- Organismos geneticamente modificados (OGMs)
- Impacto ambiental
- Uso dual (militar/civil)
- Bioterrorismo
- Propriedade intelectual

## Níveis de Biossegurança

- **BSL-1:** risco mínimo
- **BSL-2:** risco moderado
- **BSL-3:** alto risco
- **BSL-4:** risco extremo

## Regulação

Protocolos de contenção, monitoramento e aprovação ética

# Circuitos Genéticos como Dispositivos Computacionais

## Analogia com Circuitos Eletrônicos

Circuitos genéticos processam sinais bioquímicos assim como circuitos eletrônicos processam sinais elétricos

| Conceito   | Eletrônica     | Biologia         |
|------------|----------------|------------------|
| Sinal      | Voltagem       | Concentração     |
| Componente | Transistor     | Gene/Proteína    |
| Estado ON  | Alto voltagem  | Alta expressão   |
| Estado OFF | Baixo voltagem | Baixa expressão  |
| Lógica     | Portas lógicas | Regulação gênica |

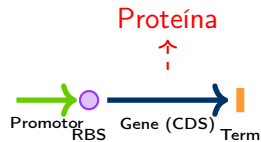
## Importante!

Princípios de design modular: componentes intercambiáveis e reutilizáveis

# BioBricks: Partes Biológicas Padronizadas

## Componentes Básicos

1. **Promotor:** controla transcrição
2. **RBS:** Ribosome Binding Site
3. **CDS:** Coding Sequence (gene)
4. **Terminador:** fim da transcrição



## Montagem Modular:

- BioBrick Assembly
- Gibson Assembly
- Golden Gate

Registry of Standard Biological Parts

<http://parts.igem.org>



# Toggle Switch: Interruptor Genético

Gardner et al. (2000)

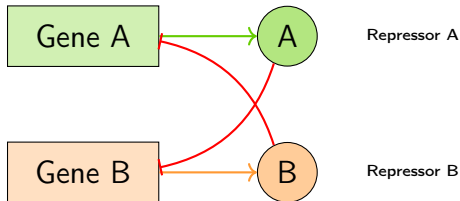
## Princípio

Dois repressores que se inibem mutuamente

- Gene A reprime gene B
- Gene B reprime gene A
- Sistema bistável

## Estados Estáveis:

- Estado 1: A alto, B baixo
- Estado 2: A baixo, B alto



**Importante!**

Primeiro circuito genético sintético com memória bistável

# Modelo Matemático do Toggle Switch

## Sistema de EDOs

$$\begin{aligned}\frac{du}{dt} &= \frac{\alpha_1}{1 + v^\beta} - u \\ \frac{dv}{dt} &= \frac{\alpha_2}{1 + u^\gamma} - v\end{aligned}$$

onde:

- $u, v$  = concentrações dos repressores
- $\alpha_1, \alpha_2$  = taxas de produção máximas
- $\beta, \gamma$  = coeficientes de Hill (cooperatividade)

**Condição para Bistabilidade:**

$$\alpha_1 \alpha_2 > (\beta + 1)(\gamma + 1)$$

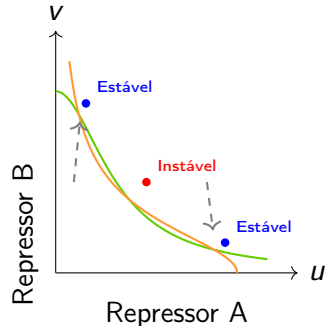
# Análise de Bistabilidade

## Pontos de Equilíbrio:

$$u = \frac{\alpha_1}{1 + v^\beta}, \quad v = \frac{\alpha_2}{1 + u^\gamma}$$

## Análise de Estabilidade:

- Jacobiano
- Autovalores
- Diagrama de bifurcação



## Resultado

Sistema pode ter 1 ou 3 pontos de equilíbrio dependendo dos parâmetros

Nullclines:  $\frac{du}{dt} = 0$  e  $\frac{dv}{dt} = 0$

# Repressilator: Oscilador Genético

Elowitz & Leibler (2000)

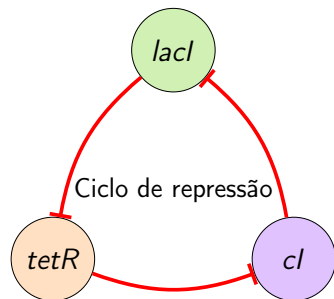
## Princípio

Três genes que se reprimem em cascata:

- Gene *lacI* reprime *tetR*
- Gene *tetR* reprime *cl*
- Gene *cl* reprime *lacI*

## Resultado:

- Oscilações periódicas
- Período  $\sim$  2-3 horas
- Visualização com GFP



**Importante!**

Primeiro oscilador genético sintético

# Modelo Matemático do Repressilator

## Sistema de EDOs

$$\begin{aligned}\frac{dm_i}{dt} &= -m_i + \frac{\alpha}{1 + p_j^n} + \alpha_0 \\ \frac{dp_i}{dt} &= -\beta(p_i - m_i)\end{aligned}$$

onde  $i = 1, 2, 3$  (lacI, tetR, cl) e  $j = i - 1 \pmod{3}$

- $m_i$  = concentração de mRNA do gene  $i$
- $p_i$  = concentração de proteína do gene  $i$
- $\alpha$  = taxa de transcrição máxima
- $n$  = coeficiente de Hill
- $\beta$  = razão de degradação proteína/mRNA

## Condições para Oscilação:

- Repressão forte ( $n$  grande)
- Tempo de resposta adequado ( $\beta$  apropriado)

# Feed-Forward Loops (FFLs)

## Motivo de Rede

Padrão comum em redes regulatórias:

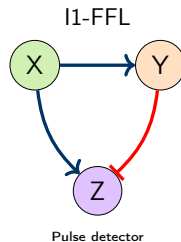
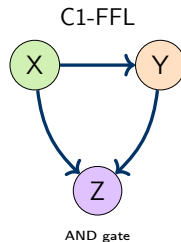
- Gene X regula Y
- Gene X regula Z
- Gene Y regula Z

## Tipos:

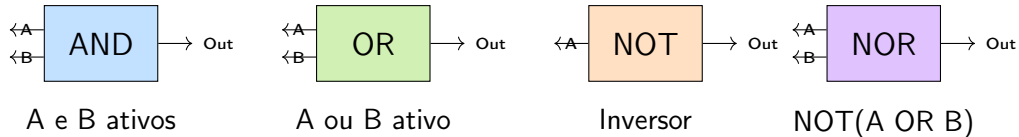
- Coherent FFL (C-FFL)
- Incoherent FFL (I-FFL)

## Funções:

- Filtro de ruído
- Detector de pulso
- Acelerador de resposta



# Portas Lógicas Genéticas



## Implementação Biológica

- **AND:** dois promotores necessários
- **OR:** promotores alternativos
- **NOT:** repressor
- **NOR:** dois repressores

# Biosensores e Repórteres

## Biosensor

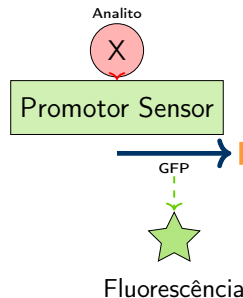
Sistema que detecta sinais químicos/físicos e produz resposta mensurável

### Componentes:

- Domínio sensor (receptor)
- Domínio de sinal
- Gene repórter

### Repórteres Comuns:

- GFP (fluorescência verde)
- RFP (fluorescência vermelha)
- Luciferase (bioluminescência)
- LacZ ( $\beta$ -galactosidase)



### Exemplo: Arsênico

Biosensor para detecção de arsênico em água usando promotor *ars* + GFP



# Simulação: Toggle Switch em Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def toggle_switch(y, t, alpha1, alpha2, beta, gamma):
6     """Modelo do toggle switch"""
7     u, v = y
8     dudt = alpha1 / (1 + v**beta) - u
9     dvdt = alpha2 / (1 + u**gamma) - v
10    return [dudt, dvdt]
11
12 # Parametros para sistema bistavel
13 alpha1 = 156.25
14 alpha2 = 15.6
15 beta = 2.5
16 gamma = 1.0
17
18 # Tempo
19 t = np.linspace(0, 50, 1000)
20
21 # Duas condicoes iniciais diferentes
22 y0_1 = [0.1, 5.0] # Estado 1
23 y0_2 = [5.0, 0.1] # Estado 2
24
25 # Resolver
26 sol1 = odeint(toggle_switch, y0_1, t, args=(alpha1, alpha2, beta, gamma))
27 sol2 = odeint(toggle_switch, y0_2, t, args=(alpha1, alpha2, beta, gamma))
28
29 # Plotar
30 plt.figure(figsize=(10, 6))
31 plt.plot(t, sol1[:, 0], 'b-', label='u (Estado 1)', linewidth=2)
32 plt.plot(t, sol1[:, 1], 'r-', label='v (Estado 1)', linewidth=2)
33 plt.plot(t, sol2[:, 0], 'b--', label='u (Estado 2)', linewidth=2)
34 plt.plot(t, sol2[:, 1], 'r--', label='v (Estado 2)', linewidth=2)
35 plt.xlabel('Tempo')
36 plt.ylabel('Concentracao')
37 plt.legend()
38 plt.title('Toggle Switch: Bistabilidade')
```

Listing 1: Modelo do toggle switch

# Simulação: Repressilator em Python

```
1 def repressilator(y, t, alpha, alpha0, beta, n):
2     """Modelo do repressilator (3 genes)"""
3     m1, p1, m2, p2, m3, p3 = y
4
5     # mRNA equations
6     dm1dt = -m1 + alpha / (1 + p3**n) + alpha0
7     dm2dt = -m2 + alpha / (1 + p1**n) + alpha0
8     dm3dt = -m3 + alpha / (1 + p2**n) + alpha0
9
10    # Protein equations
11    dp1dt = -beta * (p1 - m1)
12    dp2dt = -beta * (p2 - m2)
13    dp3dt = -beta * (p3 - m3)
14
15    return [dm1dt, dp1dt, dm2dt, dp2dt, dm3dt, dp3dt]
16
17 # Parametros
18 alpha = 216.0
19 alpha0 = 0.0
20 beta = 0.2
21 n = 2.0
22
23 # Condições iniciais
24 y0 = [0.5, 1.0, 1.5, 0.5, 2.0, 1.0]
25 t = np.linspace(0, 500, 5000)
26
27 # Resolver
28 sol = odeint(repressilator, y0, t, args=(alpha, alpha0, beta, n))
29
30 # Plotar
31 plt.figure(figsize=(12, 6))
32 plt.subplot(1, 2, 1)
33 plt.plot(t, sol[:, 1], 'g-', label='Proteína 1', linewidth=2)
34 plt.plot(t, sol[:, 3], 'orange', label='Proteína 2', linewidth=2)
35 plt.plot(t, sol[:, 5], 'purple', label='Proteína 3', linewidth=2)
36 plt.xlabel('Tempo')
37 plt.ylabel('Concentração')
38 plt.legend()
39 plt.title('Repressilator: Series Temporais')
40
41 plt.subplot(1, 2, 2)
42 plt.plot(sol[:, 1], sol[:, 3], 'b-', linewidth=1)
43 plt.xlabel('Proteína 1')
44 plt.ylabel('Proteína 2')
45 plt.title('Retrato de Fase')
```

Listing 2: Modelo do repressilator

# Engenharia de Vias Metabólicas

## Definição: Engenharia Metabólica

Modificação direcionada de vias metabólicas usando tecnologia de DNA recombinante para melhorar a produção de compostos de interesse.

### Objetivos:

- Aumentar rendimento
- Produzir novos compostos
- Remover subprodutos
- Melhorar eficiência
- Reduzir custos

### Alvos Típicos:

- Biocombustíveis (etanol, biodiesel)
- Fármacos (artemisinina, taxol)
- Aminoácidos
- Vitaminas
- Polímeros biodegradáveis

## Importante!

Combina biologia molecular, bioquímica e modelagem computacional

# Estratégias de Engenharia Metabólica

## 1. Overexpression (Superexpressão)

Aumentar expressão de enzimas limitantes da via

- Promotores fortes
- Múltiplas cópias do gene
- Otimização de códons

## 2. Gene Knockout (Deleção)

Eliminar vias competidoras ou indesejáveis

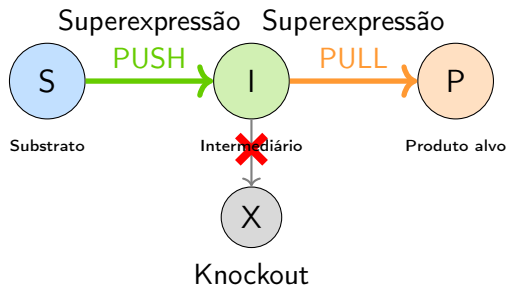
- CRISPR-Cas9
- Recombinação homóloga
- Direcionar fluxo para via de interesse

## 3. Heterologous Expression (Expressão Heteróloga)

Introduzir genes de outros organismos

- Criar novas vias
- Produzir compostos não-nativos

# Estratégia Push-Pull



## Estratégia

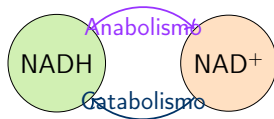
- **Push:** aumentar fluxo entrando na via
- **Pull:** aumentar conversão para produto final
- **Block:** eliminar vias competidoras

# Balanceamento de Cofatores

## Problema

Muitas reações requerem cofatores (NADH, NADPH, ATP)

- Disponibilidade limitada
- Necessário balanço entre vias
- Afeta rendimento máximo teórico



Balanço redox

## Importante!

Engenharia de cofatores: modificar especificidade de enzimas ( $\text{NADH} \leftrightarrow \text{NADPH}$ )

# Exemplo: Produção de Artemisinina

## Contexto

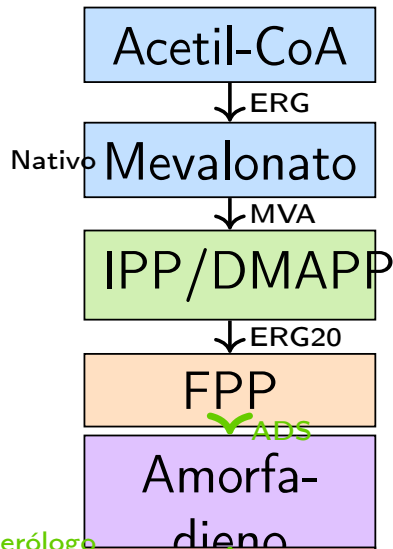
- Antimalárico potente
- Extraído de *Artemisia annua*
- Rendimento baixo (~1%)
- Alto custo

## Solução: Biologia Sintética

- Via heteróloga em *E. coli*
- Via estendida em levedura
- Expressão de genes de planta
- Otimização metabólica

## Resultado (Keasling lab):

- Produção >25 g/L
- Redução de custo 90%



# Exemplo: Produção de 1,3-Propanodiol

## Aplicação

Monômero para polímeros (PTT)

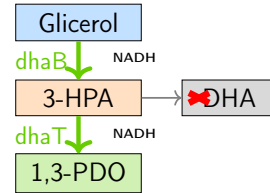
- Carpetes, plásticos
- Produção tradicional: petroquímica
- Alternativa: fermentação

## Via Sintética:

- Glicerol  $\rightarrow$  3-HPA
- 3-HPA  $\rightarrow$  1,3-PDO
- Genes de *Klebsiella*
- Expressão em *E. coli*

## DuPont/Genencor:

- Produção industrial
- Rendimento  $>135$  g/L
- Processo econômico



## Importante!

Exemplo de sucesso comercial da engenharia metabólica



# Análise de Balanço de Fluxo (FBA)

## Flux Balance Analysis

Método computacional para prever fluxos metabólicos

- Baseado em estequiometria
- Estado estacionário
- Otimização linear

Formulação:

$$\begin{aligned} &\text{maximizar} && c^T v \\ &\text{sujeito a} && Sv = 0 \\ &&& v_{min} \leq v \leq v_{max} \end{aligned}$$

onde:

- $S$  = matriz estequiométrica
- $v$  = vetor de fluxos
- $c$  = coeficientes da função objetivo (ex: biomassa)

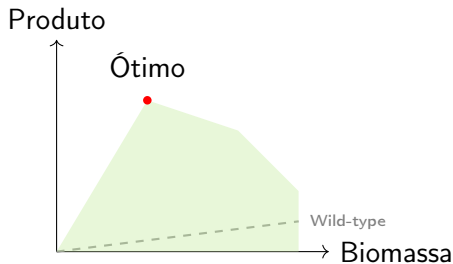
# Identificação de Alvos Genéticos

## Métodos

- **FBA**: prever fluxos
- **FVA**: análise de variabilidade
- **MOMA**: minimização de ajuste metabólico
- **OptKnock**: identificar knockouts ótimos

## OptKnock:

- Problema bi-nível
- Acoplar crescimento e produção
- Identificar deleções sinérgicas



OptKnock identifica knockouts para produção acoplada

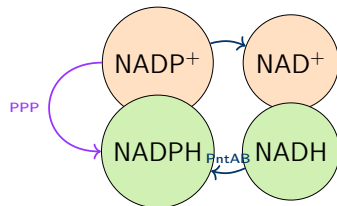
# Engenharia de Cofatores

## Estratégias

- Modificar especificidade NADH/NADPH
- Superexpressar transidrogenases
- Redirecionar fluxo de cofatores
- Usar vias alternativas

## Exemplo: NADPH

- Necessário para biossíntese
- Via das pentoses (PPP)
- Enzima málica
- Transidrogenase PntAB



## Importante!

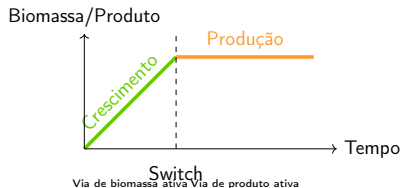
Balanço de cofatores é crítico para rendimento máximo

# Regulação Dinâmica de Vias

## Controle Temporal

Expressão gênica responsiva a sinais metabólicos

- Promotores induzíveis
- Biosensores de metabólitos
- Controle de feedback
- Evitar toxicidade de intermediários



## Exemplo: Sistema Bifásico

Fase 1: crescimento celular (promotor constitutivo)

Fase 2: produção (promotor induzível por IPTG ou temperatura)

# Engenharia Metabólica com COBRApy

```
1 import cobra
2 from cobra import Model, Reaction, Metabolite
3
4 # Carregar modelo de E. coli
5 model = cobra.io.load_json_model("e_coli_core.json")
6
7 # Realizar FBA
8 solution = model.optimize()
9 print(f"Taxa de crescimento: {solution.objective_value:.3f} h-1")
10
11 # Analisar fluxos
12 print("\nFluxos principais:")
13 for rxn_id in ['BIOMASS_Ecoli_core_w_GAM', 'EX_glc__D_e', 'EX_o2_e']:
14     rxn = model.reactions.get_by_id(rxn_id)
15     print(f"{rxn.name}: {solution.fluxes[rxn_id]:.3f}")
16
17 # Simular knockout
18 model_ko = model.copy()
19 model_ko.reactions.PGI.knock_out() # Knockout phosphoglucose isomerase
20
21 solution_ko = model_ko.optimize()
22 print(f"\nCrescimento pos-knockout: {solution_ko.objective_value:.3f} h-1")
23
24 # Análise de variabilidade de fluxo (FVA)
25 from cobra.flux_analysis import flux_variability_analysis
26
27 fva_result = flux_variability_analysis(model, reaction_list=['PGI', 'PFK'])
28 print("\nFVA:")
29 print(fva_result)
30
31 # Identificar genes essenciais
32 from cobra.flux_analysis import single_gene_deletion
33
34 deletion_results = single_gene_deletion(model)
35 essential_genes = deletion_results[deletion_results['growth'] < 0.01]
36 print(f"\nNúmero de genes essenciais: {len(essential_genes)}")
```

Listing 3: Análise FBA com COBRApy

# FSEOF: Flux Scanning com Objetivo Forçado

```
1 def fseof_analysis(model, target_reaction, biomass_fraction=0.1):
2     """
3     Flux Scanning based on Enforced Objective Flux
4     Identifica reacoes para superexpressao/delecao
5     """
6     import pandas as pd
7
8     # Forcar producao minima de biomassa
9     biomass_rxn = model.reactions.BIOMASS_Ecoli_core_w_GAM
10    max_biomass = model.optimize().objective_value
11    biomass_rxn.bounds = (biomass_fraction * max_biomass, 1000)
12
13    # Maximizar produto alvo
14    model.objective = target_reaction
15
16    results = []
17
18    # Escanear cada reacao
19    for rxn in model.reactions:
20        if rxn.id == target_reaction.id:
21            continue
22
23        # Estado original
24        original_bounds = rxn.bounds
25
26        # Testar knockout
27        rxn.bounds = (0, 0)
28        sol_ko = model.optimize()
29        flux_ko = sol_ko.objective_value if sol_ko.status == 'optimal' else 0
30
31        # Testar superexpressao (aumentar limite superior)
32        rxn.bounds = (original_bounds[0], original_bounds[1] * 10)
33        sol_oe = model.optimize()
34        flux_oe = sol_oe.objective_value if sol_oe.status == 'optimal' else 0
35
36        # Restaurar
37        rxn.bounds = original_bounds
38
39        results.append({
40            'reaction': rxn.id,
41            'knockout_effect': flux_ko,
42            'overexpression_effect': flux_oe
43        })
44
45    return pd.DataFrame(results)
46
47 # Executar FSEOF
48 target = model.reactions.EX_succ_e # Succinato como produto alvo
49 fseof_results = fseof_analysis(model, target)
50
51 # Identificar alvos promissores
52 print("Melhores alvos para superexpressao:")
53 print(fseof_results.nlargest(5, 'overexpression_effect'))
```

Listing 4: FSEOF para engenharia metabólica

# Ferramentas CAD para Biologia Sintética

## Computer-Aided Design (CAD)

Software para projetar circuitos e vias biológicas

### Ferramentas Principais:

- **Cello**: design automático de circuitos
- **j5**: montagem de DNA
- **Eugene**: linguagem de design
- **SBOLDesigner**: editor visual
- **iBioSim**: modelagem e simulação

### Funcionalidades:

- Design de construtos
- Predição de comportamento
- Otimização de parâmetros
- Seleção de partes
- Planejamento de montagem

## Cello

Entrada: especificação lógica (Verilog)

Saída: sequência de DNA otimizada

# Caracterização e Padronização de Partes

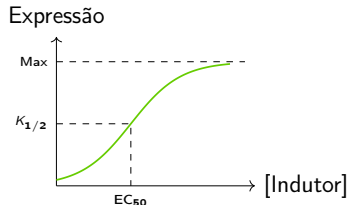
## Caracterização

Medição quantitativa de propriedades de partes biológicas

- Força de promotor (PoPS - Polymerase per Second)
- Eficiência de RBS (taxa de tradução)
- Taxa de degradação
- Resposta a indutores

## Registry of Standard Biological Parts

- >20,000 partes
- Caracterização padronizada
- Compartilhamento comunitário
- iGEM competition



Curva dose-resposta



# SBOL: Synthetic Biology Open Language

## Definição: SBOL

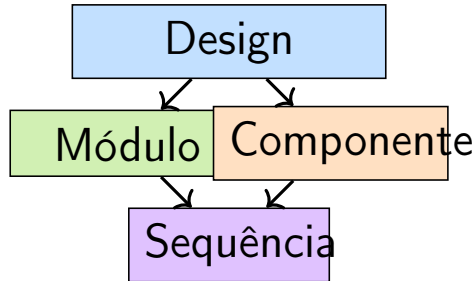
Formato de dados padronizado para representar designs de biologia sintética

### Objetivos:

- Interoperabilidade entre ferramentas
- Compartilhamento de designs
- Documentação estruturada
- Reprodutibilidade

### Componentes:

- ComponentDefinition
- ModuleDefinition
- Sequence
- Interaction



Versão atual: SBOL 3.0

## Importante!

Padrão aceito pela comunidade para intercâmbio de dados

# Frameworks de Modelagem

## Modelos Determinísticos

- **ODEs:** sistemas contínuos
- **Steady-state:** FBA
- Ferramentas: COPASI, Tellurium

## Modelos Espaciais

- **PDEs:** difusão e reação
- **Agent-based:** células individuais
- Virtual Cell

## Modelos Estocásticos

- **Gillespie:** simulação exata
- **Tau-leaping:** aproximado
- Para baixo número de moléculas

## Modelos Multi-escala

- Integrar múltiplos níveis
- Molecular → Celular → População
- CompuCell3D

## Escolha do Modelo

Depende da escala, número de moléculas e questão biológica

# Algoritmos de Otimização para Design

## OptKnock

Identifica knockouts genéticos ótimos

- Problema bi-nível
- Otimização mista inteira (MILP)
- Acopla crescimento e produção

## OptGene

Extensão usando algoritmos genéticos

- Busca heurística
- Múltiplos objetivos
- Mais flexível que OptKnock

## OptForce

Identifica intervenções mínimas

- Superexpressão e knockout
- Garante produção mínima

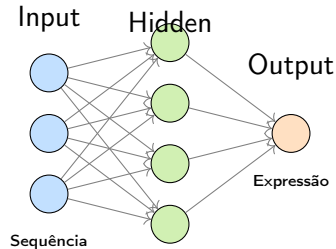
# Machine Learning para Predição de Partes

## Aplicações de ML

- Predição de força de promotor
- Otimização de RBS
- Predição de estrutura de RNA
- Design de proteínas
- Predição de interações

## Métodos:

- Regressão
- Redes neurais
- Random forests
- Deep learning



**Importante!**

ML acelera ciclo de design-build-test-learn

# Simulação Estocástica: Algoritmo de Gillespie

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gillespie_ssa(reactions, initial_state, tmax):
5     """
6     Gillespie Stochastic Simulation Algorithm
7     reactions: lista de (propensity_func, stoichiometry_change)
8     """
9     state = np.array(initial_state, dtype=float)
10    time = 0
11    times = [time]
12    states = [state.copy()]
13
14    while time < tmax:
15        # Calcular propensidades
16        propensities = np.array([prop(state) for prop, _ in reactions])
17        a0 = propensities.sum()
18
19        if a0 == 0:
20            break
21
22        # Tempo ate proxima reacao
23        tau = np.random.exponential(1/a0)
24
25        # Escolher reacao
26        r = np.random.random() * a0
27        cumsum = 0
28        for i, a in enumerate(propensities):
29            cumsum += a
30            if cumsum >= r:
31                _, change = reactions[i]
32                state += change
33                break
34
35        time += tau
36        times.append(time)
37        states.append(state.copy())
38
39    return np.array(times), np.array(states)
40
41 # Exemplo: producao e degradacao de proteina
42 k_prod = 10.0 # taxa de producao
43 k_deg = 0.5 # taxa de degradacao
44
45 reactions = [
46     (lambda s: k_prod, np.array([1])), # Producao
47     (lambda s: k_deg * s[0], np.array([-1])) # Degradacao
48 ]
49
50 times, states = gillespie_ssa(reactions, [0], tmax=20)
51 plt.plot(times, states[:, 0])
52 plt.xlabel('Tempo')
53 plt.ylabel('Numero de moleculas')
```

Listing 5: Gillespie SSA para circuitos genéticos

# Otimização de Vias com Algoritmos Genéticos

```
1 import numpy as np
2 from scipy.integrate import odeint
3
4 def pathway_model(y, t, gene_expression_levels):
5     """Modelo de via metabolica simples"""
6     S, I, P = y
7     k1, k2 = gene_expression_levels # Niveis de expressao
8
9     dSdt = -k1 * S
10    dIdt = k1 * S - k2 * I
11    dPdt = k2 * I
12
13    return [dSdt, dIdt, dPdt]
14
15 def fitness_function(gene_levels):
16     """Objetivo: maximizar produto final"""
17     y0 = [10, 0, 0] # [S, I, P]
18     t = np.linspace(0, 10, 100)
19     sol = odeint(pathway_model, y0, t, args=(gene_levels,))
20     return sol[-1, 2] # Concentracao final de P
21
22 def genetic_algorithm(pop_size=50, generations=100):
23     """Algoritmo genetico simples"""
24     # Populacao inicial
25     population = np.random.uniform(0, 10, (pop_size, 2))
26
27     for gen in range(generations):
28         # Avaliar fitness
29         fitness = np.array([fitness_function(ind) for ind in population])
30
31         # Selecao
32         sorted_idx = np.argsort(fitness)[::-1]
33         population = population[sorted_idx]
34
35         # Elitismo: manter melhores 20%
36         n_elite = pop_size // 5
37         new_pop = population[:n_elite].copy()
38
39         # Crossover e mutacao
40         while len(new_pop) < pop_size:
41             parents = population[np.random.choice(n_elite, 2, replace=False)]
42             child = (parents[0] + parents[1]) / 2 # Crossover
43             child += np.random.normal(0, 0.5, 2) # Mutacao
44             child = np.clip(child, 0, 10)
45             new_pop = np.vstack([new_pop, child])
46
47         population = new_pop[:pop_size]
48
49     return population[0]
50
51 # Executar
52 optimal_levels = genetic_algorithm()
53 print(f"Niveis otimos de expressao: {optimal_levels}")
```

Listing 6: AG para otimização de expressão gênica

# Genomas Mínimos

## Conceito

Conjunto mínimo de genes necessários para vida

- Redução sistemática do genoma
- Identificar genes essenciais
- Criar chassis simplificado

## Exemplo: JCVI-syn3.0

- *Mycoplasma mycoides*
- 531 kb (vs 1079 kb original)
- 473 genes
- Primeiro genoma mínimo sintético
- Venter Institute, 2016

## Vantagens:

- Menor carga metabólica
- Mais previsível
- Reduz interações indesejadas
- Plataforma para engenharia

## Desafios

- Crescimento lento
- Genes de função desconhecida
- Dependência de contexto

# Xenobiologia e Código Genético Expandido

## Definição: Xenobiologia

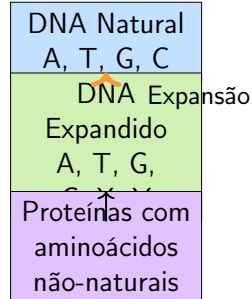
Criação de sistemas biológicos com química alternativa à base da vida natural

### Abordagens:

- Bases não-naturais (X, Y)
- Aminoácidos sintéticos
- XNA (xeno nucleic acids)
- Ribossomos ortogonais

### Aplicações:

- Contenção biológica
- Propriedades novas
- Bioortogonalidade



## Importante!

Romesberg lab: organismos com 6 letras genéticas (2017)



# Sistemas Cell-Free

## Síntese Cell-Free

Produção de proteínas e metabolismo sem células vivas

- Extratos celulares
- Componentes purificados (PURE system)
- Adicionar DNA/RNA template

## Vantagens:

- Sem viabilidade celular
- Controle preciso
- Sem membranas
- Prototipagem rápida
- Proteínas tóxicas

## Aplicações:

- Produção de proteínas
- Biossensores
- Terapêutica
- Educação
- Biologia sintética

## Produtos Comerciais

- PURExpress (NEB)
- myTXTL (Arbor Biosciences)

# Protocélulas e Células Artificiais

## Protocélulas

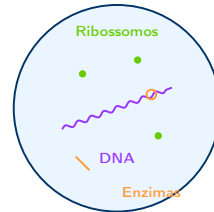
Estruturas sintéticas que mimetizam propriedades celulares

- Vesículas lipídicas
- Contêm maquinaria molecular
- Auto-replicação primitiva
- Origem da vida

### Componentes:

- Membrana (lipossomas)
- Genoma (DNA/RNA)
- Metabolismo (enzimas)
- Informação hereditária

### Protocélula



### Importante!

Objetivo de longo prazo: criar vida artificial de novo

# Sistemas Biológicos Ortogonais

## Definição: Ortogonalidade

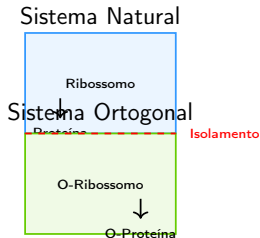
Sistemas que operam independentemente dos sistemas naturais da célula hospedeira

### Exemplos:

- Ribossomos ortogonais
- tRNAs ortogonais
- Aminoacil-tRNA sintetases
- Códon não-naturais
- Fatores de transcrição ortogonais

### Vantagens:

- Sem interferência com host
- Controle independente
- Segurança biológica



### Exemplo: Aplicação

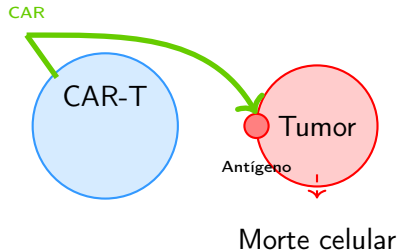
Incorporar aminoácidos não-naturais sem afetar proteoma endógeno

# Aplicações: Células Terapêuticas

## Imunoterapia CAR-T

Células T modificadas para atacar câncer

- Receptor quimérico (CAR)
- Reconhece antígeno tumoral
- Resposta imune direcionada
- FDA aprovado (2017)



## Melhorias Sintéticas:

- Switches suicidas
- Controle temporal
- Multi-targeting
- Resistência à exaustão

**Importante!**

Biologia sintética na medicina personalizada

# Aplicações: Living Sensors

## Biosensores Vivos

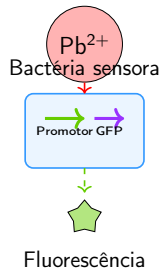
Células modificadas para detectar e responder a sinais ambientais

### Aplicações:

- Detecção de poluentes
- Diagnóstico de doenças
- Monitoramento intestinal
- Detecção de explosivos
- Controle de qualidade

### Componentes:

- Sensor (promotor responsivo)
- Processamento (circuito)
- Repórter (GFP, luz, etc)
- Memória (opcional)



## Exemplo: Probiotic Sensor

*E. coli* engineered para detectar inflamação intestinal e liberar terapêuticos

# Context Dependence e Genetic Burden

## Context Dependence

Comportamento de partes biológicas depende do contexto

- Efeitos de posição
- Disponibilidade de recursos
- Carga de expressão
- Crosstalk regulatório

### Problemas:

- Comportamento imprevisível
- Não-composicionalidade
- Necessidade de re-caracterização

## Genetic Burden

Custo metabólico da expressão heteróloga

- Ribossomos
- RNA polimerases
- Precusores (aminoácidos, NTPs)
- Energia (ATP, GTP)

### Consequências:

- Crescimento lento
- Baixa produtividade
- Instabilidade
- Seleção contra circuito

## Importante!

Design deve considerar carga metabólica e recursos compartilhados

# Estabilidade Evolutiva

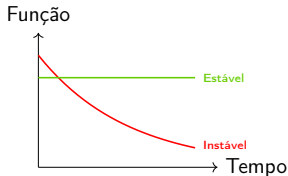
## Problema

Circuitos sintéticos podem ser perdidos ou modificados por evolução

- Pressão seletiva contra carga metabólica
- Mutações inativadoras
- Perda de plasmídeos
- Rearranjos cromossomais

## Estratégias de Estabilização:

- Integração cromossomal
- Acoplamento essencial
- Contenção genética
- Seleção positiva
- Toxina-antitoxina



**Importante!**

Trade-off entre função e fitness evolutivo

# Escalonamento: Do Laboratório à Indústria

## Desafios de Scale-up

- Condições de fermentação diferentes
- Gradientes de nutrientes e oxigênio
- Heterogeneidade populacional
- Controle de processo
- Custo de produção
- Regulação e aprovação

## Etapas:

1. Shake flask (mL)
2. Biorreator bancada (L)
3. Piloto (10-100 L)
4. Industrial (10,000+ L)

## Considerações:

- Mistura e aeração
- Controle pH e T
- Feeding strategies
- Downstream processing
- Purificação
- Custo de meios

## Exemplo: Sucesso

Produção de artemisinina e 1,3-PDO em escala industrial



# Questões Regulatórias e Éticas

## Regulação

- Organismos Geneticamente Modificados (OGM)
- Protocolos de biossegurança
- Aprovação de produtos (FDA, EMA)
- Liberação ambiental
- Rastreabilidade

## Níveis de Contenção:

- Física (laboratório)
- Biológica (auxotrofia)
- Genética (switches suicidas)

## Questões Éticas

- "Brincar de Deus"
- Impacto ecológico
- Bioterrorismo
- Acesso e equidade
- Propriedade intelectual
- Consentimento informado

## Diretrizes

- Protocolo de Cartagena
- Convenção de Armas Biológicas
- iGEM Safety Committee

# Biossegurança e Dual-Use

## Dual-Use Research

Pesquisa com potencial para uso benéfico e maléfico

- Síntese de patógenos
- Engenharia de virulência
- Resistência a antibióticos
- Toxinas

## Medidas de Segurança:

- Screening de síntese de DNA
- Controle de acesso
- Revisão ética
- Educação em biossegurança
- Códigos de conduta

## Casos Históricos:

- Síntese de poliovírus (2002)
- Influenza 1918 (2005)
- H5N1 transmissível (2011)

## Importante!

Balço entre progresso científico e segurança pública

# Direções Futuras

## AI-Driven Design

- Machine learning para predição
- Design generativo
- Otimização automática
- Análise de big data
- Modelagem multi-escala

## Foundries Automatizadas

- Robótica
- High-throughput
- Design-build-test automático
- Cloud labs
- Redução de custos e tempo

## Novas Aplicações

- Medicina regenerativa
- Agricultura sustentável
- Materiais vivos
- Computação biológica
- Terraformação
- Exploração espacial

## Visão

Biologia sintética como engenharia madura com design preditivo e confiável

## Importante!

Convergência de biologia, engenharia, computação e IA

# Exercícios - Parte 1

## Questão 1: Análise de Bistabilidade

Considere o toggle switch com parâmetros  $\alpha_1 = 100$ ,  $\alpha_2 = 20$ ,  $\beta = 2$ ,  $\gamma = 2$ .

1. Encontre os pontos de equilíbrio numericamente
2. Calcule o Jacobiano em cada ponto
3. Determine a estabilidade
4. Verifique a condição de bistabilidade:  $\alpha_1\alpha_2 > (\beta + 1)(\gamma + 1)$
5. Simule o sistema para diferentes condições iniciais

## Questão 2: Design de Biosensor

Projete um biosensor para detectar glicose:

1. Identifique promotor responsivo a glicose
2. Escolha gene repórter apropriado
3. Desenhe o construto genético (BioBrick format)
4. Estime a sensibilidade e faixa dinâmica

### Questão 3: Otimização Metabólica

Para produção de succinato em *E. coli*:

1. Use COBRApy para carregar modelo *E. coli* core
2. Maximize produção de succinato via FBA
3. Identifique genes para knockout usando FSEOF
4. Simule o efeito de knockouts
5. Calcule rendimento teórico máximo (g succinato / g glicose)

### Dica

Succinato: EX\_succ\_e

Glicose uptake: EX\_glc\_\_D\_e

# Exercícios - Parte 3

## Questão 4: Simulação Estocástica

Implemente simulação de Gillespie para repressilator:

1. Defina reações de transcrição e tradução
2. Inclua degradação de mRNA e proteína
3. Simule para 500 unidades de tempo
4. Compare com solução determinística (ODEs)
5. Analise variabilidade célula-a-célula

## Questão 5: Design de Circuito Lógico

Projete um circuito genético que implementa porta NAND:

1. Desenhe diagrama do circuito
2. Especifique componentes (promotores, genes, repressores)
3. Construa tabela verdade
4. Estime parâmetros necessários
5. Discuta possíveis problemas de implementação

# Projeto Integrador

## Projeto: Engenharia de Via para Produção de Bioplástico

Objetivo: Produzir poli-3-hidroxibutirato (PHB) em *E. coli*

### Tarefas:

#### 1. Design:

- Identificar genes necessários (phaA, phaB, phaC)
- Projetar construtos com promotores apropriados
- Planejar estratégia de integração/plasmídeo

#### 2. Modelagem:

- Construir modelo ODE da via
- Usar FBA para prever rendimento
- Identificar gargalos metabólicos

#### 3. Otimização:

- Propor knockouts para aumentar fluxo
- Balancear cofatores (NADPH)
- Simular estratégias de regulação dinâmica

#### 4. Análise:

- Calcular rendimento teórico máximo
- Estimar custos de produção
- Discutir viabilidade comercial

# Referências Principais

-  Gardner, T.S., Cantor, C.R., & Collins, J.J. (2000). Construction of a genetic toggle switch in *Escherichia coli*. *Nature*, 403, 339-342.
-  Elowitz, M.B. & Leibler, S. (2000). A synthetic oscillatory network of transcriptional regulators. *Nature*, 403, 335-338.
-  Keasling, J.D. (2010). Manufacturing molecules through metabolic engineering. *Science*, 330, 1355-1358.
-  Nielsen, J. & Keasling, J.D. (2016). Engineering cellular metabolism. *Cell*, 164, 1185-1197.
-  Cameron, D.E., Bashor, C.J., & Collins, J.J. (2014). A brief history of synthetic biology. *Nature Reviews Microbiology*, 12, 381-390.



## Livros

- Channon, K., Bromley, E.H., & Woolfson, D.N. (2008). *Synthetic Biology*. Oxford University Press.
- Smolke, C.D. (2018). *The Metabolic Pathway Engineering Handbook*. CRC Press.
- Keasling, J.D. & Venter, J.C. (2012). *Synthetic Biology*. Academic Press.

## Reviews

- Khalil, A.S. & Collins, J.J. (2010). Synthetic biology: applications come of age. *Nature Reviews Genetics*, 11, 367-379.
- Church, G.M. et al. (2014). Realizing the potential of synthetic biology. *Nature Reviews Molecular Cell Biology*, 15, 289-294.
- Lim, W.A. (2010). Designing customized cell signalling circuits. *Nature Reviews Molecular Cell Biology*, 11, 393-403.

# Recursos e Ferramentas

## Software e Ferramentas

- **COBRApy**: <https://opencobra.github.io/cobrapy/>
- **Tellurium**: <http://tellurium.analogmachine.org>
- **COPASI**: <http://copasi.org>
- **SBOLDesigner**: <https://github.com/SynBioDex/SBOLDesigner>
- **Cello**: <http://www.cellocad.org>

## Repositórios de Partes

- **iGEM Registry**: <http://parts.igem.org>
- **Addgene**: <https://www.addgene.org>
- **JBEI Registry**: <https://public-registry.jbei.org>

## Cursos Online

- MIT OpenCourseWare: Synthetic Biology
- Coursera: Bioinformatics Specialization

# Obrigado!

## Capítulo 14: Design of Biological Systems

### **Tópicos Abordados:**

Circuitos genéticos sintéticos, engenharia metabólica, ferramentas de design, células inteiras, desafios e perspectivas

*Dúvidas? Perguntas?*